# TECHNICAL REPORT

Data Storm 7.0 — Preliminary Round | Team: CodeStormers

## 1. Data Forensics and Hygiene

### 1.1 Data Landscape Assessment

We received 5 raw datasets from legacy SFA and distributor ERP systems spanning ~20,000 outlets across 4 provinces (Western, Central, North-Western, Southern) served by 10 distributors.

| Dataset | Records | Key Issues Found |
|---|---|---|
| transactions_history_final.csv | ~2M+ | Negative volumes, extreme outliers, duplicate SKU-month entries, orphan outlet IDs |
| outlet_master.csv | ~20,000 | Typos in Outlet_Type ("Grocry"), inconsistent casing, duplicate Outlet_IDs |
| outlet_coordinates.csv | ~20,000 | Coordinates outside Sri Lanka bounding box, null lat/lon |
| distributor_seasonality_details.csv | ~360 | Inconsistent seasonality labels |
| holiday_list.csv | ~100+ | Dates in ISO format with timezone, out-of-order entries |

### 1.2 Lakehouse Architecture

We implemented a 3-layer Lakehouse pipeline to ensure data integrity and traceability:

- **Bronze (Raw Ingestion):** Preserves the original state with lineage manifest (timestamp, source, file sizes).
- **Silver (Cleaned):** 10+ parameterizable DQ checks applied per dataset with full quarantine storage.
- **Gold (Enriched):** Final layer containing enriched features, peer benchmarks, and potential predictions.

### 1.3 Reusable Data Quality Library

Our pipeline utilizes six modular, parameterizable DQ functions:

- **Duplicate Check:** Composite primary key deduplication.
- **Null Check:** Mandatory field completeness validation.
- **Referential Integrity:** Cross-dataset foreign key validation.
- **Value Range:** Numeric boundary enforcement (e.g., Volume $\geq 0$, Lat $\in [5.5, 10.0]$).
- **Format/Type:** Regex-based ID validation and robust date parsing.
- **Outlier Detection:** IQR-based extreme value detection (3× IQR).

### 1.4 Quarantine Policy

**Zero data loss:** Every failed record is quarantined to `silver/quarantined/` with a `_quarantine_reason` annotation. This ensures transparency and allows for future data recovery or manual correction.

# 2. POI Data Acquisition

## 2.1 External Data Strategy

Leveraging the **OpenStreetMap Overpass API**, we enriched the internal dataset with geospatial signals to capture demand drivers that are not present in historical sales data.

| Category | OSM Tag | Relevance |
|---|---|---|
| Schools | `amenity=school` | High foot traffic; parents and students frequent nearby shops. |
| Hospitals | `amenity=hospital` | Consistent visitor traffic and 24/7 beverage demand. |
| Bus Stops | `highway=bus_stop` | Transit hubs drive high-volume impulse purchases. |

## 2.2 Feature Engineering from POIs

For each outlet, we computed proximity features including count-based metrics (radii of 500m, 1km, 2km) and a **Composite Accessibility Score**:

$$Score = 0.30 \times bus\_score + 0.35 \times school\_score + 0.35 \times hospital\_score$$

All POI queries were batched and cached to ensure API efficiency and reproducibility.

# 3. Causal Base Logic — Demand Uncapping

## 3.1 The Censoring Problem

Historical sales volume is **left-censored**: $Observed\_Volume = min(True\_Demand, Supply\_Constraint)$. To solve this, we used a multiplicative framework:

$$Potential = Base \times Seasonality \times POI\_Uplift \times Growth \times Cooler$$

## 3.2 Base Potential (Peer-Benchmarked)

We define a "Peer Group" based on `Outlet_Type × Size × Region`. The base potential is uncapped using:

$$Base = max(Own\_P95, Peer\_P90 \times 0.8, Historical\_Max)$$

## 3.3 Constraint Detection

Outlets hitting distribution ceilings are identified by a **Low CV** (Coefficient of Variation < 0.15) or a high ratio of months near their historical maximum. These outlets receive an **Uncapping Multiplier** ranging from 1.1x to 1.3x based on their gap relative to top-performing peers.

# 4. GenAI Transparency Log

Generative AI was used as an **engineering accelerator** across all phases of the project.

| Phase | AI Tool | Purpose |
|---|---|---|
| Architecture | Gemini | Lakehouse structure planning and DQ function design. |
| Code Gen | Gemini | Generating boilerplate for POI scraping and DQ scripts. |
| Methodology | Gemini | Refining the mathematical logic for demand uncapping. |

## 5. Results & Conclusions

Our analysis indicates that legacy system artifacts significantly mask true demand. By combining **Peer Benchmarking** with **External POI signals**, we have developed a robust framework to "uncap" this demand, providing a data-driven path for resource allocation and cooler deployment for January 2026.

---

Team CodeStormers — Data Storm 7.0 — Organized by Rotaract University of Moratuwa — Powered by OCTAVE